

1 Tracking scripts

There are two new scripts:

1. `process_tracking_locator_sphere.py` — main tracking script; run this on your snapshot hdf5 files to generate tracking outputs (stored in a .npz file) and a diagnostic figure that summarizes most of the outputs. It is primarily meant to track CPCs in the *rotating* case, i.e., when each condensate is close to one of the respective poles. It can also handle the non-rotating case, but it assumes one condensate per hemisphere, so if both condensates occupy, say, the North hemisphere at the same time, the results may be unsatisfactory.
2. `tracking_locator_sphere_helpers.py` — is imported by the main tracking script; contains helper functions, just in a separate script to help keep clutter in the main script to a minimum.

Using these, I highly recommend trying out tracking on data from a **rotating** simulation with a **small** value of α , but you can also start with whatever runs you already have available.

2 How to use the main script

An example job script for calling `process_tracking_locator_sphere.py` on Quest looks like:

```
#!/bin/bash
#SBATCH --account=b1094
#SBATCH --partition=ciera-himem
#SBATCH --time=04:00:00
#SBATCH --constraint=quest10
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --mem=20G
#SBATCH --job-name process-tracking-sphere
#SBATCH --mail-user=benjamin.hyatt@u.northwestern.edu
#SBATCH --mail-type=ALL
```

```
source /home/bah2659/.bashrc
d3
```

```
mpirun python process_tracking_locator_sphere.py snapshots/snapshots_s*.h5 --bin_width_phi=0.01 --alpha=
```

where in this instance I have specified an optional input argument `bin_width_phi` to be slightly smaller than the default value of 0.02 (which is set in the preamble comment at the top of `process_tracking_locator_sphere.py`).

Quest-related things that you may want or need to change:

- Path to your hdf5 output(s): this is the only *required* input argument (no `--` precedes it), as everything else has some prescribed default behavior. In this example, assumes you are running this job script inside a directory that contains the folder 'snapshots', which itself contains multiple hdf5 files of the form 'snapshots_s*.h5', with '*' being any character(s).
- Allotted job resources (`--time`, `--mem`, `--partition`, `--ntasks-per-node`, etc.):
 - The current `time` is a little bit longer than it took me to finish plotting about 20 writes, you can increase it accordingly for more writes.
 - When the `mem` line is not included, defaults to approx 3 GB (if `partition` is set to `ciera-std` and `--ntasks-per-node` is 1) – in my experience, about 10 GB is needed to plot a single write of a scalar field with resolution 2048×1024 (there is a lot of matplotlib overhead). The most available memory on a single node when `partition` is set to `ciera-std` is around 150 GB or so. If you find yourself needing more, then the `ciera-himem` partition is an option that in principle can get you closer to 1 TB (1000 GB).
 - Both of the IVP scripts currently have `max_writes=1` in their snapshots definitions, so the default settings here should work for that case – but the downfall is that *the above example doesn't parallelize by plotting each file simultaneously, it does them one at a time*. To be more efficient, you could either:

1. merge each of the `.h5` files into one `.h5` file – this could be either a “virtual” `.h5` file or one that is physically all together – you might already be doing this?
2. use a larger choice of max writes per file (e.g., `max_writes=5`) and then set `--ntasks-per-node` to be that number

For any of the above suggestions – you will likely need to consider increasing `mem` whenever you make one of the above changes, at worst proportionally to the number of writes per file, but hopefully less than that will suffice. Note: to tell if the job has enough memory, you can check the slurm output file while it is running to see if it appears to be stalled (if it is, use `sancel <jobid>` and try more memory) or you can look at `seff <jobid>` after the job is cancelled/finishes to see what percent of memory it used (if it used 100% or close to that, it probably needs more).

- Your typical Quest/conda setup: It should go without saying, but you’ll want to replace `--mail-user` with your email (or remove the lines to do with mail if you don’t want notifications), as well as the lines with `'source /home/bah2659/.bashrc'` and `'d3'` to be however you typically activate your conda environment on Quest.

Tracking options for tweaking how the tracking script performs: **the full list of options can be found in the preamble comment of `process_tracking_locator_sphere.py`** and should be well-documented and self-explanatory to some extent. But I will highlight a few that are most likely to be pertinent:

- Simulation parameters (`--gamma`, `--alpha`, `--eps`): their default values correspond to $\gamma = 2\Omega = 2 \times 1186$, $\alpha = 0.0333333$, and $\epsilon = 1$. If you are using the script on data ran with different physical parameters (e.g., non-rotating, or a different value of the friction/damping rate) you will want to specify the right value to supersede the default.
- Size of “cap” search region (`--cap_colat`, `--min_cap_colat`): the default behavior of the script will estimate the Rhines length scale, convert that to a co-latitude (latitudinal displacement from a pole), and set the edge of the search region at $2\times$ that value. Two cases in which the default behavior *automatically* changes:
 - If `--gamma` is not > 0 , then `--cap_colat`’s default behavior is overridden to set the edge of each search region at the equator.
 - If the default choice based on the Rhines scale is too small, then `--min_cap_colat` will take over by default, to prevent the search region from being too small.

Besides all of those cases, you might find that you want to specify `--cap_colat` yourself to be something specific – feel free to do so by specifying these options yourself.

- Besides these, other things you can tweak include: what time interval over which you track the CPCs (e.g., maybe you want to exclude the early transient stage before the CPCs are well-established); the size of the interval over which you measure the CPC vorticity profile; how many bins you get in your histograms; etc. Feel free to play with these options.

3 What to look for

After you’ve ran the tracking script successfully (no out-of-memory issues, etc.) I would recommend taking a look at the diagnostic figure, and assessing the situation starting with the following questions:

1. **Sanity checks: does the tracking data look reasonable?** For instance: (i) is the output cadence of the snapshots short enough to see the location of the CPC change by relatively small amounts, is it making small jumps, or is it totally erratic (output cadence too large); (ii) Are the tracked longitude values typically within the prescribed range (set by `cap_colat`) or do they look like they are very clustered close to a boundary (consider changing the size of the search region); (iii) especially if you are looking at a non-rotating case, do you think you are seeing one condensate per hemisphere?
2. **Histograms:** (i) are there enough data points to have a sense of what the distribution looks like, or does the simulation need to be ran longer? (ii) is the 2d histogram more like a ring or is it filled-in near the pole? (iii) is the 1d (marginal) φ histogram relatively uniform (this is another test for whether you have enough data points / if your output cadence is good)

3. **Relative positions:** (i) do the two condensates have similar mean latitudinal displacements from their respective poles (if simulation is rotating)? (ii) are their azimuthal phases (φ) correlated? e.g., do they have the same φ , or opposite φ (different by π), or some other constant factor? some other correlation? or are they totally uncorrelated?